Tipos de dados e variáveis

Escola Tecnológica FPFtech

Curso: Técnico em Automação Industrial

Linguagem de Programação Voltada a Microcontroladores

Semana 01

Prof. Denis Moraes Guimarães

23/09/2026

Objetivos e percurso da aula

Ao final, você deverá conseguir

Escolher tipos para representar informações de automação, declarar variáveis e acompanhar seus valores em um programa C.

1. Reconhecer tipos primitivos e suas características.
2. Declarar, inicializar e atualizar variáveis.
3. Exibir os valores e interpretar um rastreamento.
4. Resolver uma atividade guiada e quatro exercícios.

Do algoritmo aos dados

Situação: acompanhar uma estação de bombeamento

Um programa precisa registrar a quantidade de ciclos, a temperatura, o setor e a disponibilidade da estação.

Informação	Exemplo de valor
Ciclos concluídos	42
Temperatura medida	27,5 graus Celsius
Identificação do setor	A
Estação disponível	verdadeiro

Pergunta inicial

Esses quatro valores têm a mesma natureza?

O que é uma variável?

Uma variável é um objeto identificado por um nome, com um tipo e um valor armazenado durante a execução.



- O **nome** permite referenciar a informação.
- O **tipo** define valores e operações possíveis.
- O **valor** pode mudar por uma atribuição.

Tipos fundamentais usados nesta aula

Tipo em C	Representação	Exemplo de aplicação
`int`	Inteiros	Contagem de ciclos
`char`	Tipo inteiro usado também para caracteres	Identificador de setor
`float`	Valores de ponto flutuante	Medição aproximada
`double`	Ponto flutuante, precisão ao menos igual a `float`	Cálculo de engenharia
`_Bool` / `bool`	Falso ou verdadeiro	Estado de disponibilidade

`bool`, `true` e `false` estão disponíveis em C11 com `#include <stdbool.h>`.

Inteiros: quantidades sem parte fracionária

```
int ciclos = 42;
int temperatura_inteira = -5;
unsigned int pecas = 120u;
```

- `int` representa inteiros com sinal.
- `unsigned int` representa inteiros não negativos.
- `short`, `long` e `long long` são outras formas de especificar tipos inteiros.

O tipo não elimina a necessidade de validação

Escolha um tipo com capacidade para os valores esperados.
Converter um número negativo para `unsigned` não rejeita o valor.

Ponto flutuante: valores aproximados

```c
float temperatura_c = 27.5f;
double tensao_v = 3.3;
```

- No código C, a parte decimal usa **ponto**.
- O sufixo `f` especifica um literal `float`.
- Sem o sufixo, `3.3` tem tipo `double`.
- Muitos valores decimais têm representação binária aproximada; não suponha precisão infinita.

Unidade no nome

`temperatura_c` e `tensao_v` ajudam a evitar confusões entre grandezas e unidades.

Caracteres e valores lógicos

```
#include <stdbool.h>

char setor = 'A';
bool disponivel = true;
bool alarme = false;
```

- `'A'` usa aspas simples: é um caractere.
- `"A"` usa aspas duplas: é uma string, tema da próxima semana.
- `false` corresponde a 0; `true`, a 1.
- Um estado lógico descreve uma condição; o acionamento de um pino físico será estudado depois.

Tamanho e limites dependem da implementação

- Um computador e um microcontrolador podem usar tamanhos diferentes para `int`.
- `sizeof(tipo)` informa o tamanho em bytes de C.
- `sizeof(char)` é sempre 1; a quantidade de bits nesse byte é dada por `CHAR_BIT`.
- `<limits.h>` fornece limites como `INT_MIN` e `INT_MAX`.
- `<stdint.h>` oferece tipos de largura especificada. Os tipos exatos, como `uint16_t`, existem quando a implementação os suporta.

Decisão prática

Consulte o ambiente de destino; não memorize um tamanho universal para `int`.

Experimento: conhecer o ambiente de execução

```c
#include <limits.h>
#include <stdio.h>

int main(void) {
    printf("Bits por byte: %d\n", CHAR_BIT);
    printf("Bytes de int: %zu\n", sizeof(int));
    printf("Limites de int: %d a %d\n",
           INT_MIN, INT_MAX);
    printf("Bytes de double: %zu\n", sizeof(double));
    return 0;
}
```

Execute: `exemplos/02_ambiente.c`. Anote a saída e identifique quais resultados podem mudar em outra plataforma. `%zu` é o formato usado para o resultado de `sizeof`.

Declaração e inicialização

```
int ciclos = 0;
double pressao_bar = 2.4;
```

Tipo	Nome	Valor inicial
`int`	`ciclos`	`0`
`double`	`pressao_bar`	`2.4`

- Declarar informa ao compilador o nome e o tipo.
- Inicializar estabelece o primeiro valor.
- O ponto e vírgula encerra a declaração.

Atribuição: atualizar o valor armazenado

```
int ciclos = 0;
ciclos = 12;
ciclos = 18;
```

Após executar	Valor de `ciclos`
`int ciclos = 0;`	0
`ciclos = 12;`	12
`ciclos = 18;`	18

= armazena um valor. Testar igualdade usa ==, assunto da próxima aula.

Atribuição entre variáveis copia o valor

```
int leitura = 12;
int copia = leitura;
leitura = 20;
```

Etapa	`leitura`	`copia`
Primeira declaração	12	ainda não declarada
Segunda declaração	12	12
Nova atribuição	20	12

Leitura correta

`copia` recebe o valor daquele momento; ela não passa a acompanhar automaticamente `leitura`.

Escolher nomes que expliquem os dados

- Use letras, dígitos e sublinhado; não comece com dígito.
- Diferencie maiúsculas e minúsculas: `nivel` e `Nivel` são nomes distintos.
- Não use palavras reservadas, como `int`.
- Para os exemplos, prefira nomes sem acentos e em `snake_case`.

Evite	Prefira
`x`	`temperatura_c`
`2sensor`	`sensor_2`
`tempo total`	`tempo_total_ms`

Valores que devem permanecer fixos

```
const double tensao_referencia_v = 3.3;
const int capacidade_tanque_l = 500;
```

- `const` impede uma nova atribuição pelo nome da variável.
- Um nome comunica o significado de um valor de configuração.
- Inicialize o objeto `const` na declaração.

Exemplo de erro de compilação

`capacidade_tanque_l = 700;` tenta modificar um objeto declarado como `const`.

Exibir valores com printf

```
printf("Ciclos: %d\n", ciclos);
printf("Temperatura: %.1f C\n", temperatura_c);
printf("Setor: %c\n", setor);
printf("Disponivel: %d\n", disponivel);
```

Formato	Valor esperado nesta aula
`%d` / `%u`	`int` / `unsigned int`
`%c`	Caractere, promovido para `int`
`%f` / `%.1f`	Ponto flutuante / uma casa decimal

`printf` recebe `float` como `double` e `bool` como `int` pelas promoções da linguagem. `\n` inicia uma nova linha.

Exemplo completo: painel de uma estação

```c
#include <stdbool.h>
#include <stdio.h>

int main(void) {
    int ciclos = 42;
    float temperatura_c = 27.5f;
    char setor = 'A';
    bool disponivel = true;
    printf("Ciclos: %d\n", ciclos);
    printf("Temperatura: %.1f C\n", temperatura_c);
    printf("Setor: %c\n", setor);
    printf("Disponivel: %d\n", disponivel);
    return 0;
}
```

Arquivo: `exemplos/01_painel.c`. A execução usa dados simulados no computador.

Como compilar e conferir o painel

No terminal, dentro da pasta da aula:

Compilação C11

```
gcc -std=c11 -Wall -Wextra -Wpedantic
exemplos/01_painel.c -o /tmp/painel
```

Execute `/tmp/painel` e confira:

Saída esperada

```
Ciclos:  42
Temperatura:  27.5 C
Setor:  A
Disponivel:  1
```

Os avisos do compilador ajudam a encontrar incompatibilidades entre variáveis e formatos.

Conversão de tipos: o valor pode mudar

```c
int ciclos = 12;
double ciclos_reais = ciclos;    /* 12.0 */
double leitura_c = 27.9;
int leitura_inteira = (int)leitura_c; /* 27 */
```

- Uma atribuição pode converter o valor para o tipo de destino.
- O *cast* `(int)` torna explícita a conversão desejada.
- No exemplo, converter para inteiro descarta a parte fracionária; não arredonda.
- A conversão para inteiro só é válida se o resultado for representável pelo tipo de destino.

Declare antes de usar e inicialize antes de ler

```c
int main(void) {
    int ciclos = 0;
    ciclos = 8;
    return 0;
}
```

- Neste curso, começamos com variáveis locais, declaradas dentro de `main`.
- O nome fica disponível a partir da declaração até o fim do bloco em que foi declarado.
- Uma variável local automática sem inicialização não contém um valor confiável para leitura.

Regra de trabalho

Estabeleça um valor antes de consultar ou exibir a variável.

Erros comuns e como corrigi-los

Problema	Correção ou explicação
`float t = 27,5;`	Use `27.5f`; vírgula não é separador decimal em C.
`char setor = "A";`	Use aspas simples: `'A'`.
`int n; printf("%d", n);`	Inicialize `n` antes da leitura.
`printf("%d", 2.5);`	Use `%f` para esse argumento.
`int ciclos = 2.8;`	Parte fracionária perdida; reveja o tipo.

Corrija primeiro a causa indicada na mensagem do compilador; compile novamente antes de executar.

Atividade guiada: retrato de um reservatório

1. Declare `nivel_l` como inteiro, inicialmente 180.
2. Declare `temperatura_c` como `double`, inicialmente 24,5.
3. Use `'B'` para o setor e `false` para o estado de alarme.
4. Copie `nivel_l` para `nivel_anterior_l`.
5. Atualize `nivel_l` para 210 e exiba todos os valores.

Antes de executar

Quais valores devem aparecer para o nível atual e o anterior? Por quê?

Modelo resolvido: `exemplos/03_reservatorio.c`.

Conferência da atividade guiada

Variável	Valor final
`nivel_l`	210
`nivel_anterior_l`	180
`temperatura_c`	24,5
`setor`	B
`alarme`	falso (exibido como 0)

- A cópia preserva o valor anterior do nível.
- O programa armazena estados simulados: nenhum sensor ou alarme físico foi conectado.

Exercícios independentes

1. Escolha tipos para: quantidade de peças; temperatura de $-2{,}5\,^{\circ}\mathrm{C}$; setor C; presença de uma peça. Justifique.
2. Faça o rastreamento: `int a = 7; int b = a; a = 9; b = 4;`. Quais são os valores finais?
3. Escreva um programa que declare `pecas = 120`, `tensao_v = 3.3`, `setor = 'C'` e `pronto = true`; exiba cada valor com seu rótulo.
4. Corrija e explique: `double t = 21,5;`, `char s = "B";` e `int n; printf("%d", n);`.

Entrega

Respostas dos itens 1, 2 e 4; arquivo C do item 3 e sua saída. Consulte o gabarito após a tentativa.

Síntese e preparação para a próxima aula

- O tipo deve representar a natureza e a faixa dos dados.
- Declaração cria o objeto; inicialização estabelece seu primeiro valor.
- A atribuição altera o valor e pode realizar conversões.
- Nomes claros, unidades explícitas e formatos corretos tornam o programa verificável.
- Tamanhos e limites precisam ser conferidos no ambiente.

Bilhete de saída

Explique a diferença entre `int nivel = 10;` e `nivel = 20;`. Cite um dado que você representaria por `bool`.

Próxima aula: operar, comparar e combinar os valores armazenados.